

Management

FORM FOR THE SUBMISSION OF ASSESSED COURSEWORK

THIS FORM MUST BE COMPLETED AND ATTACHED TO **ALL** ASSESSED WORK.

Student Number (7 digit number)	2	7	4	4	1	0	1		
Username (2 letters & 5 digit number ab12345)	I	F	2	5	8	0	1		
Unit Code	E	F	I	M	M	0	1	3	9
Unit Title									

By submitting this work online using my unique log-in and password, I hereby confirm that this work is entirely my own. I understand that all marks are provisional until ratified by the Faculty Examination Board.

Analysing User Perceptions of Revolut: A Topic Modelling and Sentiment Analysis Approach

Table of Contents

1. Introduction	1
2. Literature Review	4
3. Data Collection and Processing	4
3.1 Data Sources and Collection	4
3.2 Data Cleaning and Preprocessing	5
4. Methodology	6
4.1 Justification	6
4.2 Sentiment Analysis	7
4.3 Topic Modelling	7
5. Analysis and Results	9
5.1 Overall Sentiment Distribution	9
5.2 Topic Level Insights	10
5.3 Platform Comparisons	12
6. Business Insights and Recommendations	13
7. Limitations	14
8. Conclusion	15
References	15
Appendix	17
Appendix A (Word Cloud)	17
Appendix B (Imports and Scrape Code)	17
Appendix C (Code for Data Cleaning, Preprocessing, Sentiment Analysis and Topic Modelling)	23
Appendix D (Code for Visualisations)	27

1. Introduction

Advancements in technology have introduced new ways for organisations to understand customer sentiment and respond to customer feedback more effectively. The growth of digital platforms has generated large amounts of user-generated content, which organisations can use to understand customer insights at scale.

In this context, social media and web analytics become major tools for extracting these insights from large volumes of unstructured and informal text data, through techniques such as topic modelling and sentiment analysis. Organisations can identify patterns in customer discussions and convert them into actionable business insights (Fan & Gordon, 2014; Chen et al., 2012).

In the financial technology (fintech) sector, services are delivered primarily through digital channels, making user trust fundamental for success. Revolut, a UK-based digital banking platform, serves 40 million customers globally, but its digital-only model has created immense debate around customer support and account security, making it an ideal subject for this study.

This report utilises Natural Language Processing techniques to help analyse user perceptions of Revolut through sentiment analysis and topic modelling. Web scraping techniques were used to scrape Reddit and Trustpilot, since Reddit provides informal debate, and Trustpilot provides more structured opinions on the platform. The raw dataset scraped comprised 3,126 documents (2,424 Reddit entries and 702 Trustpilot reviews) collected between January 2024 and March 2026. Reddit data was gathered from four different subreddits (r/Revolut, r/UKPersonalFinance, r/europersonalfinance, r/personalfinance) through public JSON endpoints, while Trustpilot data was extracted via HTML parsing. After systematic data preprocessing involving duplicate removal, spam filtering, and text normalisation, the final dataset contained 2,264 documents, representing 72.4% data retention.

This research focuses on three main questions:

1. What is the overall sentiment distribution of Revolut on social media?
2. What key themes emerge that drive the positive or negative sentiment?
3. How does sentiment for Revolut differ between Reddit and Trustpilot?

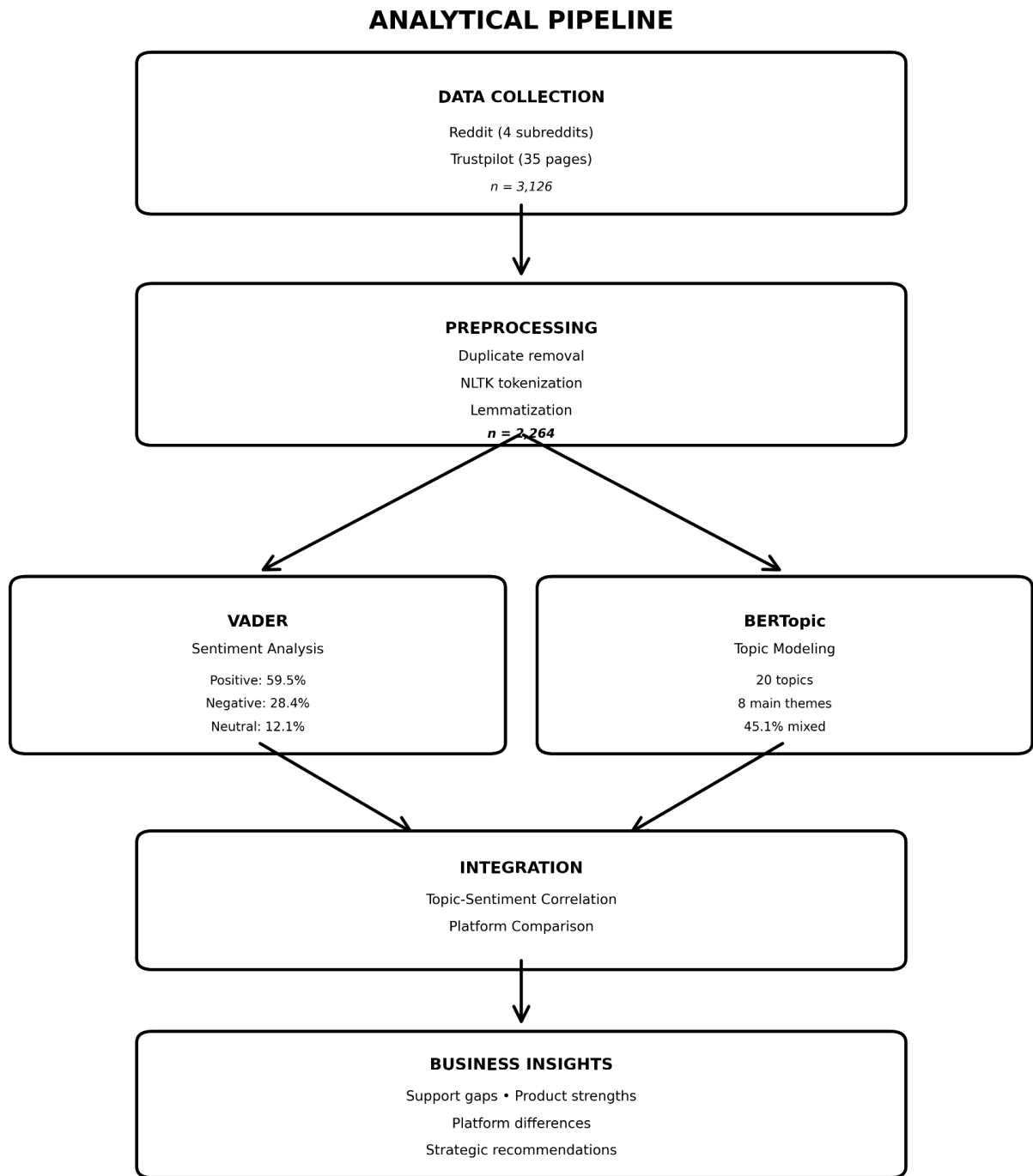


Figure 1: Complete analytical pipeline for analysing the user perception of Revolut

2. Literature Review

Previous research on social media analytics shows that there is immense value in understanding user perceptions in financial services. Tirunillai and Tellis (2012) showed that a one-standard-deviation increase in negative sentiment correlated with a 2.5% decrease in firm equity value within six weeks.

This research applies two main techniques to roughly 2200 Revolut reviews. The first is VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment analysis (Hutto & Gilbert, 2014), which is a rule-based lexicon designed for short, informal user generated text, and also includes general social media conventions such as emojis, slang, and texting grammar. It is used to classify each review as positive, negative or neutral. The second technique is topic modelling with BERTopic, it is an unsupervised machine learning technique used to discover shared topics in a body of documents. BERTopic helps simplify the process by capturing the semantic relationship between very short texts, where bag-of-words assumptions may lead to topic incoherence (Grootendorst, 2022).

Research on fintech platforms emphasises the importance of trust. Lee and Shin (2018) identified customer trust as a key success factor to establish credibility through service quality and responsive customer support for digital-only platforms. Conversely to traditional banks, where troubled customers can visit physical branches for resolution, fintech users rely solely on a digital support system, thus raising the consequences of service failures.

3. Data Collection and Processing

3.1 Data Sources and Collection

Reddit and Trustpilot were chosen as the data sources to capture different types of user content. While Trustpilot provides formal customer reviews, usually written after considerable experience, Reddit offers informal discussions where users share their good or bad experiences and seek advice from other users online.

The two Python libraries that helped develop a scraper and collect the data are BeautifulSoup and requests. No API key was required to gather data from either of these platforms. For Trustpilot, 35 pages of reviews were scraped from Revolut’s company page, extracting review text, dates and verification with 2 to 4 second delays between requests, which helped ensure ethical data collection and prevented server overload. For Reddit, data was collected from four subreddits, specifically:

1. r/Revolut
2. r/UKPersonalFinance
3. r/europersonalfinance and
4. r/personalfinance

using publicly available JSON endpoints, with 3 to 5 second delays between requests.

The initial collection gathered 3,126 texts containing 702 Trustpilot reviews and 2,424 Reddit posts and comments, complying with ethical research principles using only publicly accessible information, anonymising usernames during analysis, and following platform Terms of Service.

Table 1 summarises the data collection across both platforms, showing that Reddit contributed 2,424 documents (77.5% of raw data) while Trustpilot provided 702 reviews (22.5%).

Source	Documents	Collection Method
Reddit posts	687	JSON endpoints
Reddit comments	1737	JSON endpoints
Total Reddit	2424	-
Trustpilot	702	HTML parsing
Total	3126	-

Table 1: Data collection summary across platforms

3.2 Data Cleaning and Preprocessing

The raw scraped data was processed to ensure quality following established NLP best practices (Jurafsky & Martin, 2025). Reddit posts and comments were combined into a single dataset and

then merged with Trustpilot reviews. Data cleaning was done to remove missing values, empty texts, and exact duplicates using pandas and thus reducing the dataset from 3,126 to 2,620 documents.

Text preprocessing utilised NLTK (Natural Language Toolkit) libraries for natural language processing and converted all text to lowercase, removed URLs using singular expressions, eliminated numeric characters and punctuation, and also normalised whitespace. Tokenisation splits texts into individual words, and Stopword removal used NLTK’s English stopwords list, added with domain-specific terms which appeared throughout but contributed no additional information, such as ‘reolut’, ‘bank’, ‘account’, ‘card’ and ‘app’. Lemmatisation was also done using WordNetLemmatizer to convert inflected forms to base dictionary forms.

After data preprocessing, documents with less than three tokens were removed because they were insufficient for meaningful topic modelling. The final clean dataset comprised 2,264 texts, with Reddit and Trustpilot accounting for 93% and 7% of texts, respectively.

Table 2 illustrates the data retention through each preprocessing stage, demonstrating that 72.4% of the original dataset was retained after systematic cleaning and quality filtering.

Stage	Documents Remaining	Retention
Raw Collection	3126	100%
After duplicate & empty removal	2620	83.8%
After Text Preprocessing	2264	72.4%

Table 2: Preprocessing Pipeline

4. Methodology

4.1 Justification

The choice of VADER and BERTopic was driven by practical business requirements due to the specific characteristics of social media data. VADER was selected over supervised machine learning approaches primarily for transparency, since it classifies posts as positive or negative,

making the scoring logic interpretable. Also, supervised methods would require manually labelling thousands of social media posts to train the model.

BERTopic was chosen for topic modelling over the more traditional Latent Dirichlet Allocation (Blei, Ng & Jordan, 2003), given that LDA struggles with short, informal text common on platforms like Reddit. LDA assumes that documents contain mixtures of topics and uses the bag-of-words technique, which does not capture the semantic meaning in short social media posts (Mehrotra et al., 2013). BERTopic's transformer-based approach better understands the actual meaning and context of posts. The HDBSCAN clustering also handles the fact that not all social media content fits neatly into clear topics, allowing posts without strong thematic similarity to remain unclustered rather than forcing them into poorly fitting categories.

4.2 Sentiment Analysis

Sentiment analysis was done using VADER, which scored each document with a compound score ranging from -1.0 (extremely negative) to +1.0 (extremely positive) through lexicon-based scoring, grammatical rules, punctuation analysis, and emoji interpretation. Documents were categorised as Negative (< -0.05), Neutral (-0.05 to $+0.05$), or Positive (above $+0.05$). The overall sentiment distribution across 2,264 documents was: Positive (1,348 documents, 59.5%), Negative (643 documents, 28.4%), and Neutral (273 documents, 12.1%).

4.3 Topic Modelling

BERTopic was used to identify underlying themes through transformer-based semantic representations. The approach combined Sentence-BERT embeddings, UMAP dimensionality reduction, HDBSCAN clustering, and class-based-TF-IDF weighing (Grootendorst, 2022). Thus, enabling topic extraction from heterogeneous user-generated content.

This model identified 20 distinct topics. Topic -1 represents diverse mixed content that could not be assigned to any specific coherent themes, containing 1,021 documents (45.1%). For analytical purposes, the eight largest coherent topics (Topics 0-7) were examined in detail, collectively representing 805 documents (35.6% of the dataset). These eight topics demonstrate clear thematic coherence and sufficient document counts for reliable sentiment analysis. The

remaining smaller topics (Topics 8-19) contain fewer than 60 documents each and represent more specialised discussions.

Topic	Topic Name	Top 5 Terms	Count	% of Dataset
-1	Mixed/Diverse Content	(heterogenous themes)	1021	45.1%
0	App Technical Issues	xiaomi, app, phone, rooted, update	141	6.2%
1	Customer Support & Restriction	revolut, account, support, money, documents	131	5.8%
2	App Usability & Experience	easy, use, app, great, currencies	115	5.1%
3	Currency Exchange & Transfers	usd, account, bank, fees, wise	105	4.6%
4	Banking Comparisons	bank, use, account, main	101	4.5%
5	Account Management	banks, bank, accounts, account, people	93	4.1%
6	Travel Connectivity	esim, que, roaming, les, mobile	64	2.8%
7	Premium Features	metal, ultra, premium, subscriptions, plan	61	2.7%
8-19	Others	-	432	19.1%

Table 3: BERTopic Topic Labels and Representative Terms

5. Analysis and Results

5.1 Overall Sentiment Distribution

Sentiment analysis revealed that positive sentiment dominated the analysis with 1,348 texts (59.5%), suggesting a favourable view among users who discuss Revolut through social media channels, whereas negative sentiment remained substantial at 643 texts (28.4%), indicating persistent dissatisfaction requiring strategic attention, as can be seen in Figure 2. The neutral category of 273 texts (12.1%) also reflects users' tendency toward clear evaluative positions, consistent with social media patterns where users share content primarily while experiencing strongly positive or negative situations.

This distribution aligns well with the typical fintech patterns, where strong product features coexist with operational challenges (Lee & Shin, 2018). This substantial positive proportion suggests that Revolut successfully delivers core value propositions, such as mobile-first design, international accessibility, and competitive pricing structures. While also at the same time, the notable negative segments highlight service delivery gaps, specifically concerning McKnight, Choudhury & Kacmar's (2002) emphasis on consistent reliability as a precondition for trust.

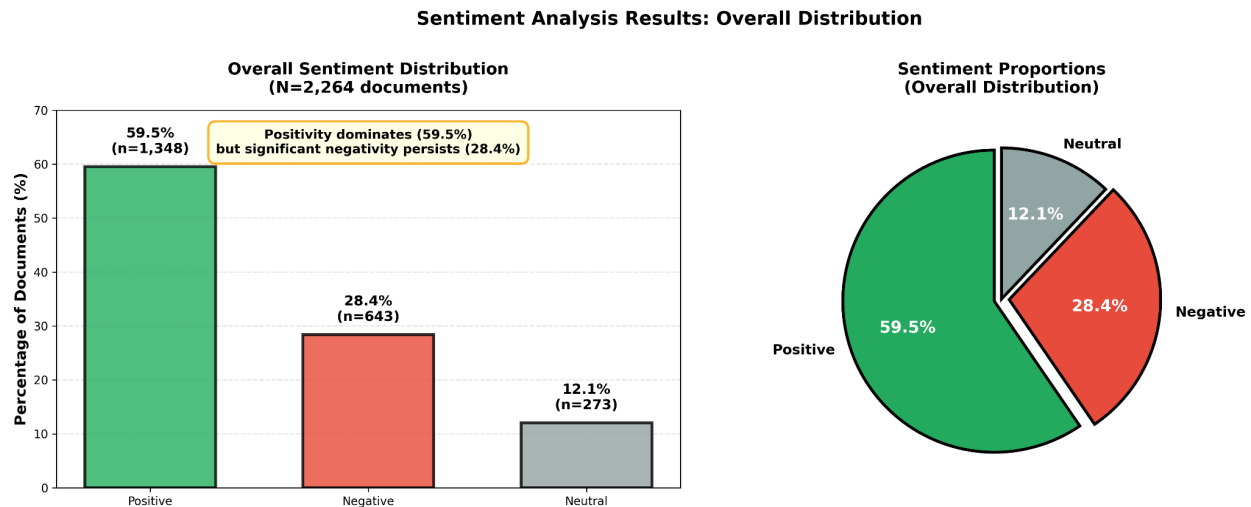


Figure 2: Overall Sentiment Distribution

5.2 Topic Level Insights

As detailed in Table 3, topic-specific sentiment analysis revealed significant diversity across service dimensions. Analysis focused on the eight largest coherent topics (Topic 0-7) collectively representing 805 documents.

Topic 2 (App Usability & Experience) exhibits overwhelmingly positive sentiment with users talking about interface design, ease of navigation, and feature accessibility. Representative terms including ‘easy’, ‘great’, ‘intuitive’, and ‘simple’ reinforce satisfaction with user experience, meeting Revolut’s mobile-first positioning. This positive performance aligns with Revolut’s strategic positioning as a mobile-first digital platform, differentiating from traditional banks through superior user experience.

Topic 3 (Currency Exchange & Transfers) shows strong positive sentiment regarding international money transfers, foreign exchange rates, and fee structures compared to traditional banks and competitors such as Wise, with users frequently mentioning concrete savings as tangible value propositions. The significance of comparison-based discourse (‘cheaper than’, ‘better rates than’) indicates that users actively compare Revolut against alternatives, with foreign exchange emerging as a clear and competitive differentiator.

Topic 1 (Customer Support & Restriction) polarised sentiment with almost equal positive and negative distribution, with some users expressing frustration regarding delayed response times, account freezes, fund access restrictions, perceived lack of human support, and escalation difficulties, while others talk about a satisfactory resolution experience, suggesting highly variable service quality rather than consistently poor performance.

Topic 0 (App Technical Issues) reveals device compatibility problems, especially affecting Xiaomi Android devices, where users report false-positive ‘rooted device’ detections preventing app functionality even with official firmware. The specificity of ‘Xiaomi’ appearing as a top representative term suggests systematic rather than random compatibility issues.

Topic 4 (Banking Comparisons) demonstrates sentiment variability as users evaluate Revolut against both neobank competitors and traditional banks. Positive evaluations emerge while comparing international functionality or fee structures, whereas more critical assessments appear regarding customer support quality or regulatory protection levels.

Topic 5 (Account Management) covers how users manage Revolut alongside other bank accounts with representative terms such as ‘banks’, ‘account’, ‘people’, suggesting users discuss multi-bank strategies rather than having Revolut as their only account. This mixed sentiment reflects different approaches, with some users appreciating having Revolut as a secondary account for purposes such as Travel or Exchange, while others express annoyance with limitations that prevent it from being their main account.

Topic 6 (Travel Connectivity) focuses on a specific product from Revolut, i.e., ‘Revolut eSim’, with terms such as ‘roaming’ and ‘eSim’ indicating discussions about using Revolut while travelling internationally, and the presence of non-English terms like ‘que’, ‘les’ suggests engagement from European users. The sentiment for this topic also remains overwhelmingly positive, with users praising the seamless eSim integration and reduced roaming issues or connectivity failures.

Topic 7 (Premium Features) talks about Revolut’s paid subscription tiers - Metal and Ultra, with users debating whether or not the premium features are worth the cost. The sentiment remains mixed, with some appreciating benefits like lounge access and cashback offers, while others

point out that competitors offer similar features on their free accounts. Thus implying that Revolut might be struggling to justify its premium pricing against competitors.

Figure 3 presents sentiment distribution across the eight main topics, revealing the stark contrast between the highly positive topics (App Usability, Currency Exchange, Travel Connectivity) and more polarised areas (Customer Support, Account Management).

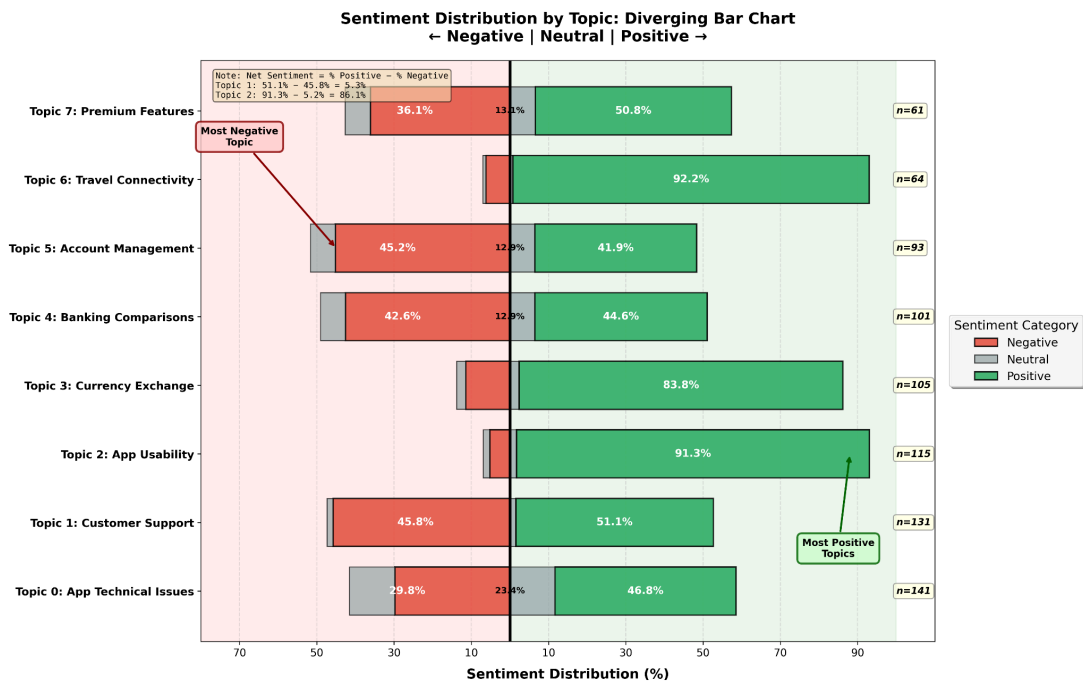


Figure 3: Sentiment Distribution by Topic

5.3 Platform Comparisons

The sentiment distribution was found to be distinct between Reddit and Trustpilot since Trustpilot reviews reveal significantly higher positive sentiment proportions, whereas Reddit discussions involve negative sentiment concentrations, as is visible in Figure 4.

This divergence signifies platform-specific behavioural patterns as Trustpilot's structured review format and public visibility create incentives for balanced evaluations, and Reddit's informal conversational format eases problem exploration and advice-seeking, naturally attracting users

experiencing difficulties. Both patterns help provide valid complementary insights. While Trustpilot captures overall satisfaction assessments, Reddit reveals specific problem areas and early warning signals.

Taking into consideration the limited Trustpilot sample, the results should be interpreted as indicative rather than definitive, with Reddit data providing a stronger representation of user conversation patterns.

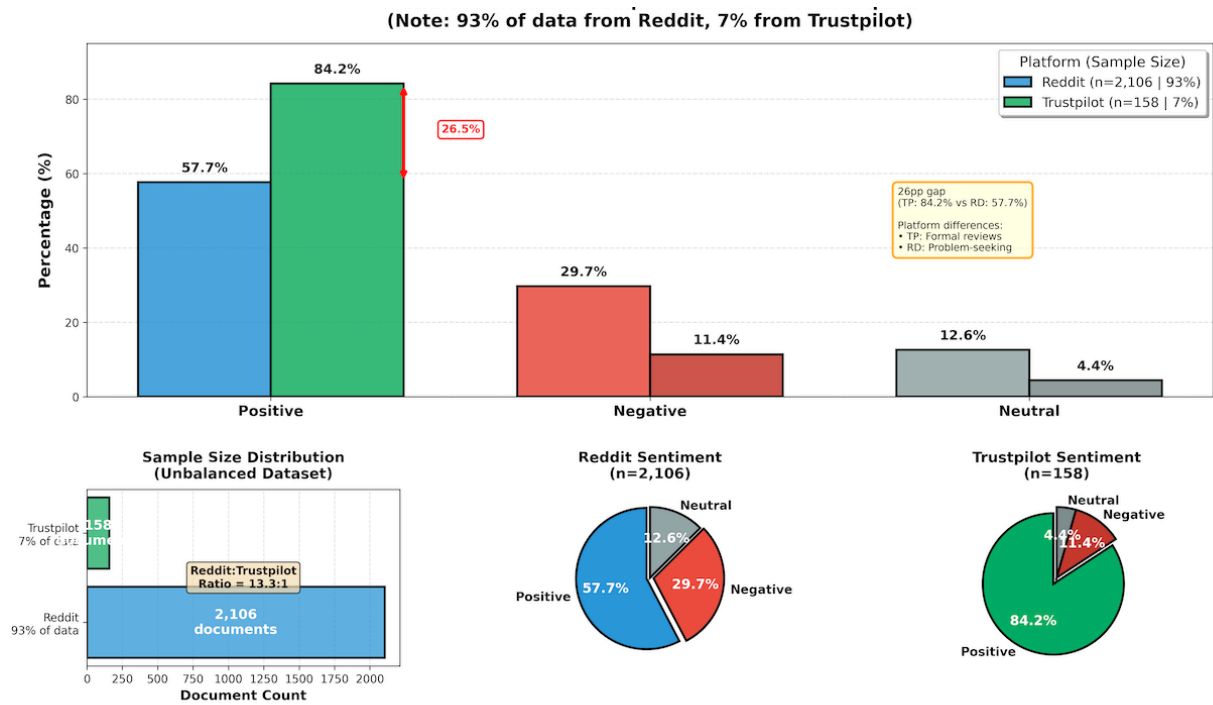


Figure 4: Platform Comparison

6. Business Insights and Recommendations

Through these findings, an interesting pattern emerges where overall sentiment appears to be reasonably positive, yet closer examination reveals notable variation across different service areas. Revolut seems to perform well in almost all aspects while facing small challenges in the operational aspects of service delivery.

Several approaches could help address these concerns. Implementing better communication during account review processes would reduce user uncertainty, since people tend to be less frustrated when they know what is happening and when to expect resolution. Creating clearer pathways for urgent issues, such as those affecting account access, could help ensure serious problems get higher priority and establishing clearer expectations around response times for different types of issues would also help manage user expectations better.

The technical compatibility issues around certain Android devices require working with affected manufacturers to identify what is triggering security flags immediately. Also, providing more information to users when compatibility issues occur would help them understand what is happening rather than just seeing the generic error messages.

7. Limitations

Multiple limitations affect this study. Selection bias exists because only users who actively post online are represented, likely over-representing extreme experiences, particularly negative ones. Platform bias is also present as Reddit users tend to be younger and more tech-savvy than the general banking population, while Geographic bias favours UK and European users. These limitations mean the findings may not fully generalise to Revolut's entire 40 million user base.

While both Trustpilot and Reddit showed distinct sentiment patterns, the small sample size of the Trustpilot data limited the statistical power of comparisons between platforms. The small sample size also means that these findings should be interpreted with caution rather than as definitive evidence of platform-specific behaviour.

Methodologically, BERTopic's density-based clustering identified 45.1% of documents as mixed content without a clear thematic association, limiting topic-specific analyses. Future research could employ aspect-based sentiment analysis to handle mixed-sentiment documents better, and also apply the identical methodology to competitor platforms using comparative studies.

8. Conclusion

This study helps demonstrate the value of social media analytics for understanding user perceptions in the fintech industry. Analysis of 2,264 user-generated texts from Trustpilot and Reddit identified distinct service dimensions and quantified sentiment through systematic application of BERTopic modelling and VADER sentiment analysis.

Key findings show that while overall sentiment is moderately positive, significant dissatisfaction still exists in customer support, concentrated with demonstrated strengths in app usability and currency exchange services.

The central insight is that though Revolut excels in features such as currency exchange, operational excellence in service delivery is equally critical for building and maintaining customer trust.

References

- Blei, D., Ng, A. and Jordan, M. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, [online] 3(993-1022), pp.993–1022. Available at: <https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf>
- Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business Intelligence and Analytics: From Big Data to Big Impact. *MIS Quarterly* 1 December 2012; 36 (4): 1165–1188. <https://doi.org/10.2307/41703503>
- Fan, W. and Gordon, M.D. (2014). The power of social media analytics. *Communications of the ACM*, [online] 57(6), pp.74–81. doi:<https://doi.org/10.1145/2602574>
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794 [cs]. [online] doi:<https://doi.org/10.48550/arXiv.2203.05794>
- Hutto, C. and Gilbert, E. (2014). VADER: a Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. *Proceedings of the International AAAI Conference on Web and Social Media*, [online] 8(1). doi:<https://doi.org/10.1609/icwsm.v8i1.14550>
- Jurafsky, D. and Martin, J.H. (2025). *Speech and Language Processing*. [online] Stanford.edu. Available at: <https://web.stanford.edu/~jurafsky/slp3/>


```

import numpy as np
import requests
from bs4 import BeautifulSoup
import time
import random
from datetime import datetime
import json
import re
import ast
!pip install vaderSentiment
!pip install bertopic

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from bertopic import BERTopic

import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from collections import Counter

# download NLTK data if needed
try:
    nltk.download('punkt', quiet=True)
    nltk.download('stopwords', quiet=True)
    nltk.download('wordnet', quiet=True)
except:
    pass

# Reddit scraper - uses public JSON endpoints so no API key needed
class RedditScraper:

    def __init__(self):
        self.headers = {
            'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36'
        }
        self.posts_data = []

```

```

        self.comments_data = []

        def scrape_subreddit(self, subreddit_name, search_query=None,
limit=100):
            all_posts = []
            after = None

            while len(all_posts) < 500:
                try:
                    if search_query:
                        url =
f"https://www.reddit.com/r/{subreddit_name}/search.json"
                        params = {
                            'q': search_query,
                            'sort': 'relevance',
                            't': 'year',
                            'limit': limit,
                            'restrict_sr': 'on'
                        }
                    else:
                        url =
f"https://www.reddit.com/r/{subreddit_name}/top.json"
                        params = {'t': 'year', 'limit': limit}

                    if after:
                        params['after'] = after

                    response = requests.get(url, headers=self.headers,
params=params, timeout=15)

                    if response.status_code == 200:
                        data = response.json()
                        posts = data['data']['children']

                        if not posts:
                            break

                        for post in posts:
                            post_data = post['data']
                            text = post_data.get('selftext', '').strip() or
post_data.get('title', '')

```

```

        all_posts.append({
            'post_id': post_data['id'],
            'title': post_data.get('title', ''),
            'text': text,
            'author': post_data.get('author', 'unknown'),
            'score': post_data.get('score', 0),
            'num_comments': post_data.get('num_comments',
0),
            'date':
datetime.fromtimestamp(post_data.get('created_utc',
0)).strftime('%Y-%m-%d'),
            'subreddit': subreddit_name,
            'source': 'Reddit'
        })

        after = data['data'].get('after')
        if not after:
            break

        elif response.status_code == 429:
            time.sleep(10)
            continue
        else:
            break

    except Exception as e:
        break

    time.sleep(random.uniform(2, 4))

    self.posts_data.extend(all_posts)
    return len(all_posts)

    def scrape_comments(self, max_posts=40, max_comments=50):
        sorted_posts = sorted(self.posts_data, key=lambda x:
x['num_comments'], reverse=True)
        posts_to_scrape = sorted_posts[:max_posts]

        for i, post in enumerate(posts_to_scrape):
            try:

```

```

        url = f"https://www.reddit.com{post['title']}.json"
        if 'reddit.com' in post.get('title', ''):
            url = post['title'] + '.json'

        response = requests.get(url, headers=self.headers,
timeout=15)

        if response.status_code == 200:
            data = response.json()
            if len(data) > 1:
                comments = data[1]['data']['children']

                for comment in comments[:max_comments]:
                    try:
                        comment_data = comment['data']

                        if comment_data.get('body') in ['[deleted]',
'[removed]', None]:
                            continue
                        if comment_data.get('author') ==
'AutoModerator':
                            continue

                        self.comments_data.append({
                            'comment_id': comment_data['id'],
                            'text': comment_data.get('body', ''),
                            'author': comment_data.get('author',
'unknown'),

                            'score': comment_data.get('score', 0),
                            'date':
datetime.fromtimestamp(comment_data.get('created_utc',
0)).strftime('%Y-%m-%d'),

                            'subreddit': post['subreddit'],
                            'source': 'Reddit'
                        })
                    except:
                        continue

            except:
                continue

        time.sleep(random.uniform(1, 3))

```

```

# Trustpilot scraper
class TrustpilotScraper:

    def __init__(self, company_name="revolut"):
        self.company_name = company_name
        self.base_url =
f"https://www.trustpilot.com/review/www.{company_name}.com"
        self.headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'
        }
        self.reviews = []

    def scrape_reviews(self, max_pages=35):
        for page_num in range(1, max_pages + 1):
            try:
                url = f"{self.base_url}?page={page_num}"
                response = requests.get(url, headers=self.headers,
timeout=10)

                if response.status_code != 200:
                    break

                soup = BeautifulSoup(response.content, 'html.parser')

                # find review cards
                review_cards = soup.find_all('article', {'class': lambda
x: x and 'review' in x.lower()})
                if not review_cards:
                    review_cards = soup.find_all('div',
{'data-service-review-card-paper': True})

                if not review_cards:
                    break

                for card in review_cards:
                    try:
                        # get rating
                        rating_elem = card.find('div', {'class': lambda x:
x and 'star-rating' in x.lower()})
                        rating = None

```

```

        if rating_elem:
            rating_img = rating_elem.find('img')
            if rating_img and 'alt' in rating_img.attrs:
                rating_text = rating_img['alt']
                rating = int(rating_text.split()[1]) if
'Rated' in rating_text else None

        # get text
        text_elem = card.find('p', {'class': lambda x: x
and 'typography_body' in x.lower()})
        text = text_elem.text.strip() if text_elem else ""

        # get date
        date_elem = card.find('time')
        date = date_elem['datetime'] if date_elem and
'datetime' in date_elem.attrs else None

        if text:
            self.reviews.append({
                'text': text,
                'rating': rating,
                'date': date,
                'source': 'Trustpilot'
            })
        except:
            continue

    except Exception as e:
        break

    time.sleep(random.uniform(2, 4))

return self.reviews

```

Appendix C (Code for Data Cleaning, Preprocessing, Sentiment Analysis and Topic Modelling)

```

# load the collected data
reddit_posts = pd.read_csv('revolut_reddit_posts.csv')

```

```

reddit_comments = pd.read_csv('revolut_reddit_comments.csv')
trustpilot = pd.read_csv('revolut_trustpilot_reviews.csv')

# combine reddit posts and comments
reddit_text = pd.concat([reddit_posts['text'], reddit_comments['text']],
ignore_index=True)
trustpilot_text = trustpilot['text']

# create datasets with source labels
df_reddit = pd.DataFrame({'text': reddit_text, 'source': 'Reddit'})
df_trustpilot = pd.DataFrame({'text': trustpilot_text, 'source':
'Trustpilot'})

# merge both
df = pd.concat([df_reddit, df_trustpilot], ignore_index=True)

# remove missing and empty values
df = df.dropna(subset=['text'])
df = df[df['text'].str.strip() != ""]
df = df.drop_duplicates(subset=['text'])
df = df.reset_index(drop=True)

# preprocessing
import re

stop_words = set(stopwords.words('english'))

# add custom stopwords
custom_stopwords = {
    "revolut", "bank", "account", "card", "money",
    "use", "using", "also", "one", "get", "got",
    "would", "could", "even", "really",
    "im", "ive", "dont", "cant", "didnt"
}

stop_words = stop_words.union(custom_stopwords)

lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = str(text).lower()

```

```

        text = re.sub(r'http\S+|www\S+', '', text)
        text = re.sub(r'\d+', '', text)
        text = re.sub(r'^\w\s', '', text)
        text = re.sub(r'\s+', ' ', text).strip()
        return text

def preprocess_text(text):
    tokens = text.split()
    tokens = [word for word in tokens if word not in stop_words and
len(word) > 2]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return tokens

# apply preprocessing
df['clean_text'] = df['text'].apply(clean_text)
df['tokens'] = df['clean_text'].apply(preprocess_text)

# remove empty rows
df = df[df['tokens'].map(len) > 3]
df = df.drop_duplicates(subset=['clean_text'])
df = df.reset_index(drop=True)

# save
df.to_csv('cleaned_all_data.csv', index=False)

```

```

# load preprocessed data
df = pd.read_csv('revolut_preprocessed.csv')

# sentiment analysis with VADER
def perform_sentiment_analysis(df):
    analyzer = SentimentIntensityAnalyzer()

    sentiment_scores = []
    for text in df['text']:
        scores = analyzer.polarity_scores(text)
        sentiment_scores.append(scores['compound'])

    df['sentiment_score'] = sentiment_scores

    # classify as positive/negative/neutral
    def classify_sentiment(score):

```

```

        if score >= 0.05:
            return 'Positive'
        elif score <= -0.05:
            return 'Negative'
        else:
            return 'Neutral'

    df['sentiment'] = df['sentiment_score'].apply(classify_sentiment)

    return df

# run sentiment analysis
df = perform_sentiment_analysis(df)

# save with sentiment
df.to_csv('revolut_with_sentiment.csv', index=False)

```

```

# SENTIMENT ANALYSIS

# load cleaned data
df = pd.read_csv('cleaned_all_data.csv')

# convert tokens column back to list (pandas saves it as string)
df['tokens'] = df['tokens'].apply(lambda x: eval(x) if isinstance(x, str)
else x)

# sentiment analysis
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

def get_sentiment(text):
    score = analyzer.polarity_scores(text)['compound']
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

df['sentiment'] = df['text'].apply(get_sentiment)

```

```

# save
df.to_csv('data_with_sentiment.csv', index=False)

# TOPIC MODELLING

# load data with sentiment
df = pd.read_csv('data_with_sentiment.csv')

# convert tokens back
df['tokens'] = df['tokens'].apply(lambda x: eval(x) if isinstance(x, str)
else x)

# bertopic
from bertopic import BERTopic

# prepare documents
documents = [' '.join(tokens) for tokens in df['tokens']]

# fit model
topic_model = BERTopic(
    nr_topics=20,
    min_topic_size=30,
    calculate_probabilities=False,
    verbose=False
)

topics, probs = topic_model.fit_transform(documents)
df['bertopic'] = topics

# save final dataset
df.to_csv('final_dataset.csv', index=False)

# save topic info
topic_model.get_topic_info().to_csv("bertopic_topics.csv", index=False)

```

Appendix D (Code for Visualisations)

```

# visualization 4: sentiment volume matrix (quadrant chart)

```

```

def create_sentiment_volume_matrix(df,
output_path='sentiment_volume_matrix.png'):

    # calculate metrics by topic
    topic_sentiment =
df[df['bertopic'].isin(range(8))].groupby(['bertopic',
'sentiment']).size().unstack(fill_value=0)

    topic_metrics = pd.DataFrame()
    for topic in range(8):
        if topic not in topic_sentiment.index:
            continue
        total = topic_sentiment.loc[topic].sum()
        pos_pct = (topic_sentiment.loc[topic, 'Positive'] / total * 100)
        neg_pct = (topic_sentiment.loc[topic, 'Negative'] / total * 100)

        topic_metrics.loc[topic, 'net'] = pos_pct - neg_pct
        topic_metrics.loc[topic, 'docs'] = total
        topic_metrics.loc[topic, 'pct'] = total / len(df) * 100

    # topic names
    topic_names = {
        0: 'App Technical\nIssues',
        1: 'Customer\nSupport',
        2: 'App\nUsability',
        3: 'Currency\nExchange',
        4: 'Banking\nComparisons',
        5: 'Account\nManagement',
        6: 'Travel\nConnectivity',
        7: 'Premium\nFeatures'
    }

    x_values = topic_metrics['pct'].values
    y_values = topic_metrics['net'].values
    sizes = topic_metrics['docs'].values * 4

    colors = ['#27ae60' if net > 30 else '#2ecc71' if net > 0 else
'#e67e22' if net > -30 else '#e74c3c'
               for net in y_values]

    fig, ax = plt.subplots(figsize=(14, 10))

```

```

# scatter plot
scatter = ax.scatter(x_values, y_values, s=sizes, c=colors,
                    alpha=0.7, edgecolors='black', linewidth=2.5,
zorder=3)

# add labels
for i, (x, y) in enumerate(zip(x_values, y_values)):
    if i == 1:
        offset_x, offset_y = -15, -25
        fontsize, bbox_color = 11, '#ffcccc'
        arrow = True
    elif i in [2, 3]:
        offset_x, offset_y = 10, 15 if i == 2 else -25
        fontsize, bbox_color = 10, '#ccffcc'
        arrow = True
    else:
        offset_x, offset_y = 8, 8
        fontsize, bbox_color = 9, 'white'
        arrow = False

    ax.annotate(f"Topic {i}\n{topic_names[i]}",
                xy=(x, y),
                xytext=(offset_x, offset_y),
                textcoords='offset points',
                fontsize=fontsize,
                fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.4', facecolor=bbox_color,
                        edgecolor='black', alpha=0.8, linewidth=1.5),
                arrowprops=dict(arrowstyle='->',
connectionstyle='arc3,rad=0.2',
                                color='black', lw=1.5) if arrow else
None,
                zorder=4)

# quadrant lines
median_prevalence = np.median(x_values)
ax.axhline(y=0, color='#34495e', linestyle='-', linewidth=2.5,
alpha=0.8, zorder=1)
ax.axvline(x=median_prevalence, color='#34495e', linestyle='-',
linewidth=2.5, alpha=0.8, zorder=1)

```

```

# quadrant labels
quadrant_style = dict(fontsize=13, ha='center', va='center',
fontweight='bold')

ax.text(5.5, 70, 'LEVERAGE\nOPPORTUNITIES',
        bbox=dict(boxstyle='round,pad=1', facecolor='#d5f4e6',
                    edgecolor='#27ae60', alpha=0.7, linewidth=3),
        **quadrant_style, color='#27ae60', zorder=2)

ax.text(5.5, -50, 'URGENT\nACTION REQUIRED',
        bbox=dict(boxstyle='round,pad=1', facecolor='#fadbd8',
                    edgecolor='#e74c3c', alpha=0.7, linewidth=3),
        **quadrant_style, color='#c0392b', zorder=2)

ax.text(3.2, 70, 'NICHE\nSTRENGTHS',
        bbox=dict(boxstyle='round,pad=1', facecolor='#fef9e7',
                    edgecolor='#f39c12', alpha=0.7, linewidth=3),
        **quadrant_style, color='#d68910', zorder=2)

ax.text(3.2, -50, 'MONITOR',
        bbox=dict(boxstyle='round,pad=1', facecolor='#eaecee',
                    edgecolor='#7f8c8d', alpha=0.7, linewidth=3),
        **quadrant_style, color='#566573', zorder=2)

ax.set_xlabel('Topic Prevalence (% of Dataset)', fontsize=14,
fontweight='bold', labelpad=10)
ax.set_ylabel('Net Sentiment (% Positive - % Negative)',
fontsize=14, fontweight='bold', labelpad=10)
ax.set_title('Strategic Topic Prioritization Matrix\nPrevalence vs
Sentiment Analysis',
             fontsize=16, fontweight='bold', pad=20)

ax.set_xlim(2.2, 6.8)
ax.set_ylim(-75, 90)
ax.grid(True, alpha=0.25, linestyle=':', linewidth=1, zorder=0)
ax.set_axisbelow(True)

# legend
from matplotlib.lines import Line2D
legend_elements = [

```

```

        Line2D([0], [0], marker='o', color='w', label='Strongly Positive
(net >30%)',
               markerfacecolor='#27ae60', markersize=12,
               markeredgecolor='black', markeredgewidth=2),
        Line2D([0], [0], marker='o', color='w', label='Moderately Positive
(net 0-30%)',
               markerfacecolor='#2ecc71', markersize=12,
               markeredgecolor='black', markeredgewidth=2),
        Line2D([0], [0], marker='o', color='w', label='Mixed (net -30 to
0%)',
               markerfacecolor='#e67e22', markersize=12,
               markeredgecolor='black', markeredgewidth=2),
        Line2D([0], [0], marker='o', color='w', label='Negative (net
<-30%)',
               markerfacecolor='#e74c3c', markersize=12,
               markeredgecolor='black', markeredgewidth=2),
    ]

    ax.legend(handles=legend_elements, loc='center left',
bbox_to_anchor=(1.02, 0.5),
               frameon=True, fancybox=True, shadow=True, fontsize=10,
               title='Net Sentiment', title_fontsize=11, borderaxespad=0)

    ax.text(0.02, 0.02, 'Note: Bubble size represents document count',
            transform=ax.transAxes, fontsize=10,
            ha='left', va='bottom', style='italic',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5,
            pad=0.5))

    ax.annotate(f'Topic 1: High visibility\n+ severe negativity\n=
Critical priority',
               xy=(x_values[1], y_values[1]), xytext=(4.8, -25),
               fontsize=9, ha='center',
               bbox=dict(boxstyle='round', facecolor='#ffe6e6',
               edgecolor='red', alpha=0.8, linewidth=2),
               arrowprops=dict(arrowstyle='->', color='red', lw=2.5,
               connectionstyle='arc3,rad=-0.3'))

    plt.tight_layout()
    plt.savefig(output_path, dpi=300, bbox_inches='tight',
facecolor='white')

```

```

plt.close()

create_sentiment_volume_matrix(df)

# visualization 5: word clouds for positive vs negative
def create_word_clouds(df, output_path='sentiment_wordclouds.png'):

    # parse tokens from string format
    def parse_tokens(token_str):
        try:
            return eval(token_str) if isinstance(token_str, str) else
token_str
        except:
            return []

    # get tokens
    positive_tokens = df[df['sentiment'] ==
'Positive']['tokens'].apply(parse_tokens)
    negative_tokens = df[df['sentiment'] ==
'Negative']['tokens'].apply(parse_tokens)

    # flatten lists
    positive_words = []
    for token_list in positive_tokens:
        if isinstance(token_list, list):
            positive_words.extend(token_list)

    negative_words = []
    for token_list in negative_tokens:
        if isinstance(token_list, list):
            negative_words.extend(token_list)

    positive_text = ' '.join(positive_words)
    negative_text = ' '.join(negative_words)

    # additional stopwords
    additional_stopwords = [
'revolut', 'bank', 'account', 'card', 'app', 'use', 'using', 'used',
'one', 'get', 'got', 'like', 'would', 'make', 'really', 'much',
'people', 'also', 'still', 'even', 'say', 'said', 'want', 'need',
'payment', 'transfer', 'service', 'transaction', 'customer', 'go',

```

```

'could', 'two', 'three', 'first', 'new', 'back', 'made', 'going',
'year', 'month', 'day', 'time', 'week', 'hour', 'fund', 'know',
'thing', 'way', 'anyone', 'anything', 'every', 'since', 'amount',
'phone', 'business', 'reason', 'without', 'pay', 'paying', 'paid',
'money', 'fee', 'charge'
]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 7))

# positive word cloud
wc_positive = WordCloud(
width=800,
height=600,
background_color='white',
colormap='Greens',
stopwords=set(additional_stopwords),
max_words=50,
relative_scaling=0.6,
min_font_size=10,
random_state=42,
collocations=False,
prefer_horizontal=0.7
).generate(positive_text)

ax1.imshow(wc_positive, interpolation='bilinear')
ax1.axis('off')
ax1.set_title('Positive Sentiment Documents\n(n=1,348 | 59.5%)',
              fontsize=16, fontweight='bold', color='darkgreen',
pad=15)

for spine in ['top', 'bottom', 'left', 'right']:
ax1.spines[spine].set_edgecolor('darkgreen')
ax1.spines[spine].set_linewidth(3)
ax1.spines[spine].set_visible(True)

# negative word cloud
wc_negative = WordCloud(
width=800,
height=600,
background_color='white',
colormap='Reds',

```

```

stopwords=set(additional_stopwords),
max_words=50,
relative_scaling=0.6,
min_font_size=10,
random_state=42,
collocations=False,
prefer_horizontal=0.7
).generate(negative_text)

ax2.imshow(wc_negative, interpolation='bilinear')
ax2.axis('off')
ax2.set_title('Negative Sentiment Documents\n(n=643 | 28.4%)',
             fontsize=16, fontweight='bold', color='darkred', pad=15)

for spine in ['top', 'bottom', 'left', 'right']:
    ax2.spines[spine].set_edgecolor('darkred')
    ax2.spines[spine].set_linewidth(3)
    ax2.spines[spine].set_visible(True)

fig.suptitle('Linguistic Markers of Sentiment: Positive vs Negative
User Discourse',
            fontsize=18, fontweight='bold', y=0.98)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(output_path, dpi=300, bbox_inches='tight',
facecolor='white')
plt.close()

create_word_clouds(df)

```